1. (3 points) Consider the number of the syllables in the pronunciation of the integer n. For example, when $n = 1$, the pronunciation is "one" and there is one syllable. For each of the following, give the tightest/simplest function possible, or state that there is no function possible.

   (a) Give a function $f(n)$ so that the number of syllables in the pronunciation of n is $O(f(n))$ for all $n \geq 1$.

   Solution: For large n, integers in English are pronounced in groups of three digits (hundreds, thousands, millions, billions, etc.). For example, the number $999,999,999$ is pronounced as "nine hundred ninety-nine million, nine hundred ninety-nine thousand, nine hundred ninety-nine". Each group of three digits has a constant maximum number of sylalbles, and the number of groups of three digits is proportional to $\log_{10} n$. So, the tightest upper bound on the number of syllables in the pronunciation of n is $O(\log n)$. (So $f(n) = \log n$.)

   ■

   (b) Give a function $g(n)$ so that the number of syllables in the pronunciation of n is $\Omega(g(n))$ for all $n \geq 1$.

   Solution: Even as n grows large, each whole multiple of 1000 has a short name (million, billion, trillion, etc.). Therefore, the tightest lower bound on the number of syllables in the pronunciation of n is $\Omega(1)$. (So $g(n) = 1$.)

   ■

   (c) Give a function $h(n)$ so that the number of syllables in the pronunciation of n is $\Theta(h(n))$ for all $n \geq 1$.

   Solution: Since the tightest upper bound is $O(\log n)$ and the tightest lower bound is $\Omega(1)$, there is no function $h(n)$ so that the number of syllables in the pronunciation of n is $\Theta(h(n))$ for all $n \geq 1$. ■

2. (3 points) Prove that $\log_a n = \Theta(\log_b n)$ for any positive constants $a \neq 1, b$ by proving each of the following. You must use the definition of big-O and big-Omega in your proofs; that is, you must give a c and an n_0. (Hint: Use the change of base formula for logarithms, which states that $\log_a n = \frac{\log_b n}{\log_b a}$.)

   (a) $log_a n = O(log_b n)$

   Solution: To show that $\log_a n = O(\log_b n)$, we need to find constants $c > 0$ and $n_0 \geq 1$ such that for all $n \geq n_0$, the following inequality holds:

   $$\log_a n \leq c \cdot \log_b n.$$

   Let $c = \frac{1}{\log_b a}$. Since $a > 0$ and $a \neq 1$, we have $\log_b a > 0$, so $c > 0$.
   Now notice that

   $$\log_a n = \frac{\log_b n}{\log_b a} \qquad \text{(change of base formula)}$$

$$= \frac{1}{\log_b a} \cdot \log_b n \qquad \text{(rewriting)}$$
$$= c \cdot \log_b n \qquad \text{(substituting } c\text{)}.$$

So it must be the case that $\log_a n \leq c \cdot \log_b n$ for all $n \geq 1$, so we can choose $n_0 = 1$ (or any positive integer), and the inequality holds for all $n \geq n_0$. Therefore, $\log_a n = O(\log_b n)$.

■

(b) $log_a n = \Omega(log_b n)$

Solution: In (a), we showed that choosing $c = \frac{1}{\log_b a}$ implies that $\log_a n = c \cdot \log_b n$ for all $n \geq 1$. So we can choose the same c and still have $\log_a n \geq c \cdot \log_b n$ for all $n \geq 1$, and so we have given a valid choice of c and n_0 to show that $\log_a n = \Omega(\log_b n)$. ■

3. (4 points) For each of the following algorithms, you will analyze both the worst and best case runtimes.

(a)
```
IsPrime(n):
    if n < 2
        return False
    for d = 2 to ⌊√n⌋
        if n mod d = 0
            return False
    return True
```

i. Describe a best-case input to IsPrime.

Solution: We need to identify the class of inputs for which the algorithm runs in the least amount of time. Notice that for any even n, the algorithm will return false after a constant number of steps. ■

ii. Give a function $f(n)$ so that the best-case runtime of IsPrime is $\Theta(f(n))$.

Solution: $f(n) = 1$.

■

iii. Describe a worst-case input to IsPrime.

Solution: A worst-case input for IsPrime is any prime number, since the algorithm will have to check all possible divisors up to $\lfloor \sqrt{n} \rfloor$ before returning True.

■

iv. Give a function $f(n)$ so that the worst-case runtime of IsPrime is $\Theta(f(n))$.

Solution: $f(n) = \sqrt{n}$. ■

(b)

```
HasDuplicate(A[1..n]):
  for i = 1 to n
      for j = i + 1 to n
          if A[i] = A[j]
              return True
  return False
```

i. Describe a best-case input to HasDuplicate.

Solution: A best-case input for HasDuplicate is an array where the first two elements are equal, since the algorithm will return True after a constant number of steps. ■

ii. Give a function $f(n)$ so that the best-case runtime of HasDuplicate is $\Theta(f(n))$.

Solution: $f(n) = 1$. ■

iii. Describe a worst-case input to HasDuplicate.

Solution: A worst-case input for HasDuplicate is an array where all elements are distinct, since the algorithm will have to check all pairs of elements before returning False. ■

iv. Give a function $f(n)$ so that the worst-case runtime of HasDuplicate is $\Theta(f(n))$.

Solution: $f(n) = n^2$. ■